

Stack mit AVR-Assembler

Allgemein

- Der Stack (oder Stapel) ist eine Datenstruktur. Datenstrukturen verwalten und organisieren Daten.
- Der Stack arbeitet nach dem Prinzip Last-In-First-Out (LIFO). D.h. der letzte Eintrag der aufgenommen wurde, wird auch als erstes wieder entfernt.
- Andere Beispiele für Datenstrukturen sind Listen (LIFO oder FIFO) und Queues (FIFO). FIFO steht hier für First-In-First-Out. Also der erste gesetzte Eintrag in die Datenstruktur, wird auch als erstes wieder entfernt.

Befehle

- Zwei grundlegende Operationen wurden als fertige Assembler-Befehle umgesetzt:

- push

- z.B. Wert aus Register 16 an oberste Stelle in den Stack kopieren
- Beispiel-Code:

```
push r16
```

- pop

- z.B. Wert aus oberster Stelle vom Stack in Register 16 laden und Stack-Pointer um eine Stelle verringern, sodass Wert beim nächsten push, rcall etc. überschrieben werden kann. Eine Löschung findet noch nicht statt.
- Beispiel-Code:

```
pop r16
```

Arbeitsweise

- Intern funktioniert die Navigation mit Stack-Pointer. Pointer (bzw. Zeiger) dienen zur Abspeicherung von Speicheradressen, an denen der eigentliche Wert liegt. Register oder Variablen hingegen speichern normalerweise bereits die später zu nutzenden Werte.

- Speicheradresse: (Darstellungsproblem bei 16-bit auf 8-bit Register, daher sog. Byte-Aufteilung: vordere Teil als High-Byte und hintere als Teil Low-Byte)
 - Beispiel Assembler-Funktionen zum Aufteilen des Ende der Speicheradresse vom Arbeitsspeicher (16bit auslesbar über RAMEND):
 - Beispiel-Code, um vorderen Teil (High-Byte) des Ende vom Arbeitsspeichers auszulesen. Im Nachgang wird dieser Wert in das Register SPH (Stack-Pointer High-Byte) überführt:

```
HIGH(RAMEND)
```

- Beispiel-Code, um hinteren Teil (Low-Byte) des Ende vom Arbeitsspeichers auszulesen. Im Nachgang wird dieser Wert in das Register SPL (Stack-Pointer Low-Byte) überführt:

```
LOW(RAMEND)
```

- Umsetzung in AVR-Assembler: Werte aus 2x 8-bit-Register SPH & SPL, die zusammen auf eine 16-bit Speicheradresse verweisen. Die Anwendung sehen wir im nächsten Abschnitt.

Limitierung vom Stack überwinden

- über X-,Y-,Z-Pointer können Werte aus beliebigen Speicheradressen aus dem Arbeitsspeicher ausgelesen und sogar an andere Stellen kopiert werden. Der Z-Pointer kann darüber hinaus auch auf dem EEPROM und den Flash-Speicher jeweils lesend zugreifen.
- In der Theorie zeigt Peek den Wert des obersten Elementes eines Stacks ohne ihn zu ändern oder zu löschen. Dies ist nicht als Assembler-Funktion umgesetzt. Im weiteren Verlauf ist eine mögliche Umsetzung als Unterprogramm (Subroutine).

Verfolgung von Änderungen im Stack

Grundzustand

Zuerst muss der Start für den Stack-Pointer definiert werden. Die notwendigen Werte lassen sich über RAMEND auslesen. Da RAMEND aus einer 16-bit Speicheradresse besteht, muss diese mittels der Funktionen LOW() bzw. HIGH() entsprechend aufgeteilt

werden. In diesem Beispiel dient Register 16 als Zwischenspeicher für die Werte. Diese werden mittels out in das IO-Register SPH und SPL geladen.

Beispiel-Code

```
ldi r16, HIGH(RAMEND)
out SPH, r16
ldi r16, LOW(RAMEND)
out SPL, r16
```

Übersicht Arbeitsspeicher

Adresse	Wert dezimal	Wert hexadezimal	Stack-Pointer	SPH	SPL
0x0100	0	0x00			
...					
0x08FC	0	0x00			
0x08FD	0	0x00			
0x08FE	0	0x00			
0x08FF	0	0x00	←	0x08	0xFF

Zustand nach 1. push

Der Wert 1 wird über den Befehl ldi in das Register 16 eingespeichert und dann wird mittels push der Wert im Register 16 oben auf den Stack gelegt. Da es noch keine weiteren Einträge gibt, landet der Wert auf dem Ende des Arbeitsspeichers (0x08FF).

Beispiel-Code

```
ldi r16, 1
push r16
```

Übersicht Arbeitsspeicher

Adresse	Wert dezimal	Wert hexadezimal	Stack-Pointer	SPH	SPL
0x0100	0	0x00			
...					
0x08FC	0	0x00			
0x08FD	0	0x00			
0x08FE	0	0x00	←	0x08	0xFE
0x08FF	1	0x01			

Übersicht Register

Register	Wert dezimal	Wert hexadezimal
r0	0	0x00
...		
r16	1	0x01
...		
r31	0	0x00

Zustand nach 2. push

Der Wert 2 wird über den Befehl ldi in das Register 17 eingespeichert und dann wird mittels push der Wert im Register 17 oben auf den Stack gelegt. Da es bereits einen weiteren Eintrag gibt, landet der Wert über den letzten Eintrag (0x08FE).

Beispiel-Code

```
ldi r17, 2
push r17
```

Übersicht Arbeitsspeicher

Adresse	Wert dezimal	Wert hexadezimal	Stack-Pointer	SPH	SPL
0x0100	0	0x00			
...					
0x08FC	0	0x00			
0x08FD	0	0x00	←	0x08	0xFD
0x08FE	2	0x02			
0x08FF	1	0x01			

Übersicht Register

Register	Wert dezimal	Wert hexadezimal
r0	0	0x00
...		
r16	1	0x01
r17	2	0x02
...		
r31	0	0x00

Zustand nach 3. push

Der Wert 3 wird über den Befehl ldi in das Register 18 eingespeichert und dann wird mittels push der Wert im Register 18 oben auf den Stack gelegt. Da es bereits zwei weitere Einträge gibt, landet der Wert über den letzten Eintrag (0x08FD).

Beispiel-Code

```
ldi r18, 3
push r18
```

Übersicht Arbeitsspeicher

Adresse	Wert dezimal	Wert hexadezimal	Stack-Pointer	SPH	SPL
0x0100	0	0x00			
...					
0x08FC	0	0x00	←	0x08	0xFC
0x08FD	3	0x03			
0x08FE	2	0x02			
0x08FF	1	0x01			

Übersicht Register

Register	Wert dezimal	Wert hexadezimal
r0	0	0x00
...		
r16	1	0x01
r17	2	0x02
r18	3	0x03
...		
r31	0	0x00

Zustand nach pop

Mit dem Befehl pop wird der letzte Eintrag aus dem Stack in das Register 19 geladen. Gleichzeitig wird der Stack-Pointer um eins erhöht, sodass er auf den vorletzten Eintrag zeigt. Der letzte Eintrag wird aber noch nicht gelöscht, sondern wird bei der nächsten Ablage auf den Stack (z.B. push oder rcall) überschrieben.

Beispiel-Code

```
pop r19
```

Übersicht Arbeitsspeicher

Adresse	Wert dezimal	Wert hexadezimal	Stack-Pointer	SPH	SPL
0x0100	0	0x00			
...					
0x08FC	0	0x00			
0x08FD	3	0x03	←	0x08	0xFD
0x08FE	2	0x02			
0x08FF	1	0x01			

Übersicht Register

Register	Wert dezimal	Wert hexadezimal
r0	0	0x00
...		
r16	1	0x01
r17	2	0x02
r18	3	0x03
r19	3	0x03
...		
r31	0	0x00

Zustand nach Aufruf eines Unterprogrammes

Peek als Unterprogramm

`Peek` könnte als Unterprogramm implementiert werden. Ziel ist es, **den Wert des obersten Eintrags im Stack anzuzeigen, ohne ihn zu verändern oder zu löschen**. Der ausgelesene Wert wird anschließend in **Register 24** ausgegeben.

Z-Pointer vorbereiten

Damit der **Z-Pointer** auf die richtige Speicheradresse zeigt, müssen zunächst die beiden Bytes der Adresse in die entsprechenden Register geladen werden:

- **Register 31 (ZH)**: High-Byte
- **Register 30 (ZL)**: Low-Byte

Anschließend kann der Z-Pointer mithilfe eines Offsets auf die gewünschte Speicheradresse verschoben werden.

Position des Stack-Pointers

Der **Stack-Pointer** zeigt auf die Speicheradresse direkt unterhalb des letzten Eintrags. Daher liegt die Adresse des Stack-Pointers um **1 Byte unterhalb** der Adresse des obersten Stack-Eintrags.

Aus diesem Grund muss die Adresse um **1 erhöht** werden:

```
Z + 1
```

Dadurch zeigt der Z-Pointer auf den tatsächlichen obersten Eintrag des Stacks.

Verhalten von `rcall`

Der Befehl `rcall` legt die **Rücksprungadresse** auf dem Stack ab. Diese besteht aus dem **High- und Low-Byte des Program Counters (PC)** und zeigt auf den Befehl, der direkt auf `rcall` folgt.

Dadurch wird der Stack-Pointer um **2 Byte** verringert, anstatt wie bei `push` nur um 1 Byte.

In diesem Beispiel wird der Stack-Pointer dadurch auf:

0x08FB

gesetzt.

Da das High-Byte der Rücksprungadresse hier `0x00` entspricht, wird dieses vermutlich nicht als sichtbarer zusätzlicher Stack-Eintrag berücksichtigt bzw. ist für die Betrachtung des gespeicherten Wertes nicht relevant.

Rückkehr mit `ret`

Nach dem Erreichen des Befehls `ret` wird die zuvor gespeicherte Rücksprungadresse vom Stack gelesen. Anschließend wird der **Stack-Pointer wieder auf den Zustand vor dem Aufruf des Unterprogramms** zurückgesetzt.

Damit bleibt der eigentliche Inhalt des Stacks durch `peek` unverändert.

Warum `0xFF` in Register 24?

Der Wert

0xFF

wird zunächst in **Register 24** gespeichert. Das dient dazu, dass auch der Fall erkannt werden kann, bei dem das **High-Byte des obersten Stack-Eintrags** `0x00` ist.

Würde Register 24 beispielsweise vorher bereits `0x00` enthalten, wäre nicht eindeutig erkennbar, ob tatsächlich `0x00` aus dem Stack gelesen wurde oder ob das Register einfach unverändert geblieben ist.

Zusammengefasst: `peek` liest den obersten Stack-Eintrag aus, ohne den Stack-Inhalt zu verändern oder den Eintrag zu entfernen. Der gelesene Wert wird in **Register 24** zurückgegeben.

Beispiel-Code als Unterprogramm

Peek:

```
in    r30, SPL
in    r31, SPH
ldd   r24, Z+1
ret
```

Aufruf des Unterprogramms

```
rcall Peek
```

Gesamter Code für Beispiel

```
; PC = 0x0000  
ldi r16, HIGH(RAMEND) ; Oberes Byte der RAM-Endadresse laden  
  
; PC = 0x0001  
out SPH, r16 ; Oberes Stackpointer-Register setzen  
  
; PC = 0x0002  
ldi r16, LOW(RAMEND) ; Unteres Byte der RAM-Endadresse laden  
  
; PC = 0x0003  
out SPL, r16 ; Unteres Stackpointer-Register setzen  
; → Stack zeigt jetzt auf das Ende des RAMs  
  
; PC = 0x0004  
ldi r16, 1 ; Wert 1 in r16 laden  
  
; PC = 0x0005  
push r16 ; 1 auf den Stack legen  
  
; PC = 0x0006  
ldi r17, 2 ; Wert 2 in r17 laden  
  
; PC = 0x0007  
push r17 ; 2 auf den Stack legen  
  
; PC = 0x0008  
ldi r18, 3 ; Wert 3 in r18 laden  
  
; PC = 0x0009  
push r18 ; 3 auf den Stack legen  
; Stack (oben → unten): 3, 2, 1  
  
; PC = 0x000A  
pop r19 ; Obersten Stackwert (3) nach r19 holen  
  
; PC = 0x000B  
pop r20 ; Nächsten Stackwert (2) nach r20 holen  
; Auf dem Stack verbleibt nur noch die 1  
  
; PC = 0x000C
```

```

ldi r24, 255                ; r24 mit 255 vorbelegen

; PC = 0x000D
rcall Peek                  ; Rücksprungadresse auf den Stack legen
                             ; und Unterprogramm Peek aufrufen

; PC = 0x000E
Ende:
rjmp Ende                   ; Endlosschleife

; PC = 0x000F
Peek:
    ; PC = 0x000F
    in r30, SPL              ; Aktuelles Low-Byte des Stackpointers nach

    ; PC = 0x0010
    in r31, SPH              ; Aktuelles High-Byte des Stackpointers nach
                             ; Z zeigt jetzt auf den aktuellen Stackpoin

    ; PC = 0x0011
    ldd r24, Z+1              ; Wert an Adresse SP+1 lesen (Peek ohne Pop)
                             ; Rücksprungadresse liegt auf SP/SP+1,
                             ; der gelesene Wert hängt vom Stackaufbau a

    ; PC = 0x0012
    ret                      ; Rücksprungadresse vom Stack holen
                             ; und zum Aufrufer zurückkehren

```

Übersicht Arbeitsspeicher

Adresse	Wert dezimal	Wert hexadezimal	Stack-Pointer	SPH	SPL
0x0100	0	0x00			
...					
0x08FC	0	0x00			
0x08FD	0	0x00			
0x08FE	14	0x0E	←	0x08	0xFE
0x08FF	1	0x01			

Übersicht Register

Register	Wert dezimal	Wert hexadezimal
r0	0	0x00
...		
r16	1	0x01
r17	2	0x02
r18	3	0x03
r19	3	0x03
r20	2	0x02
...		
r24	0	0x00
...		
r30	252	0xFC
r31	8	0x08

Anhang

Einheiten

Nutzung	Bezeichnung	15	14	13	12	11	10	9	8
kleinste Informationseinheit	1 Bit								
kleinste adressierbare Speichereinheit	1 Byte = 8 Bit								
hintere Teil der Speicheradresse	Low-Byte								

Nutzung	Bezeichnung	15	14	13	12	11	10	9	8
vordere Teil der Speicheradresse	High-Byte								
Speicheradresse	2 Byte = 16 Bit = 1 Word								

Speicherkapazität für Stack/SRAM berechnen

1. Startadresse

Die Startadresse ist:

```
0x1000
```

2. Endadresse

Die Endadresse setzt sich aus `0x08` und `0xFF` zusammen:

```
0x08FF
```

Das entspricht dezimal:

```
0x08FF = 2303
```

3. Speicherkapazität berechnen

Die Anzahl der Bytes berechnet sich mit:

```
Endadresse - Startadresse + 1
```

Also:

```
0x08FF - 0x0800 + 1
= 0x0800
= 2048 Byte
```

4. Ergebnis

2048 Byte = 2 kByte

→ Die Speicherkapazität beträgt 2 kByte.

Speicherbereiche

Adressbereich	Speicherbereich	Beschreibung
0x0000– 0x001F	Register R0–R31	32 Arbeitsregister der CPU (kein SRAM)
0x0020– 0x005F	I/O-Register	Standard-I/O-Register (z. B. PORTB , DDRB , PINB , SREG)
0x0060– 0x00FF	Erweiterte I/O- Register	Zusätzliche I/O-Register (z. B. Timer, UART, ADC)
0x0100– 0x08FF	SRAM	Datenspeicher für Variablen, Heap und Stack

Simulator

- Project -> Projektname Properties -> Tool -> Selected debugger/programmer -> Simulator
- Add or remove buttons -> Windows
 - Processor Status: Alle Register anzeigen lassen
 - Memory -> Memory1 -> data IRAM: Arbeitsspeicher anzeigen
 - Memory -> Memory1 -> prog FLASH: Flash-Speicher anzeigen

Program Counter

- Projektordner / Debug / Projektname.lss

Ausschnitt:

```
000004 e001                                ; PC = 0x0004
ldi r16, 1                                ; Wert 1 in r16 laden
```

.. 0004 entspricht dem Program Counter für den Befehl in der entsprechenden Zeile

X-,Y-,Z-Pointer

Zeiger	Registerpaar	Direkter Offset +q	Beispiel
X	R27:R26	✗ Nein	LD R16, X
Y	R29:R28	✓ Ja	LDD R16, Y+5
Z	R31:R30	✓ Ja	LDD R16, Z+5